

ranges::fold

Document #: P2322R2
Date: 2021-04-14
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Return Type](#)
 - [3.1 Always return T](#)
 - [3.2 Return the result of the initial invocation](#)
- [4 Other fold algorithms](#)
 - [4.1 fold first](#)
 - [4.2 fold right](#)
 - [4.3 Naming](#)
- [5 Wording](#)
- [6 References](#)

§ 1 Revision History

[P2322R1] used *weakly-assignable-from* as the constraint, this elevates it to `assignable_from`. This revision also changes the return type of `fold` to no longer be the type of the initial value, see [the discussion](#).

[P2322R0] used `regular_invocable` as the constraint in the *foldable* concept, but we don't need that for this algorithm and it prohibits reasonable uses like a mutating operation. `invocable` is the sufficient constraint here (in the same way that it is for `for_each`). Also restructured the API to overload on providing the initial value instead of having differently named algorithms.

§ 2 Introduction

As described in [P2214R0], there is one very important rangified algorithm missing from the standard library: `fold`.

While we do have an iterator-based version of `fold` in the standard library, it is currently named `accumulate`, defaults to performing `+` on its operands, and is found in the header `<numeric>`. But `fold` is

much more than addition, so as described in the linked paper, it's important to give it the more generic name and to avoid a default operator.

Also as described in the linked paper, it is important to avoid over-constraining `fold` in a way that prevents using it for heterogeneous folds. As such, the `fold` specified in this paper only requires one particular invocation of the binary operator and there is no `common_reference` requirement between any of the types involved.

Lastly, the `fold` here is proposed to go into `<algorithm>` rather than `<numeric>` since there is nothing especially numeric about it.

§ 3 Return Type

Consider the example:

```
std::vector<double> v = {0.25, 0.75};
auto r = ranges::fold(v, 1, std::plus());
```

What is the type and value of `r`? There are two choices, which I'll demonstrate with implementations (with incomplete constraints).

§ 3.1 Always return τ

We implement like so:

```
template <range R, movable T, typename F>
auto fold(R&& r, T init, F op) -> T
{
    ranges::iterator_t<R> first = ranges::begin(r);
    ranges::sentinel_t<R> last = ranges::end(r);
    for (; first != last; ++first) {
        init = invoke(op, move(init), *first);
    }
    return init;
}
```

Here, `fold(v, 1, std::plus())` is an `int` because the initial value is `1`. Since our accumulator is an `int`, the result here is `1`. This is consistent with `std::accumulate` and is simple to reason about and specify. But it is also a common gotcha with `std::accumulate`.

Note that if we use `assignable_from<T&, invoke_result_t<F&, T, range_reference_t<R>>>` as the constraint on this algorithm, in this example this becomes `assignable_from<int&, double>`. We would be violating the semantic requirements of `assignable_from`, which state 18.4.8 [\[concept.assignable\]/1.5](#):

(1.5) After evaluating `lhs = rhs`:

(1.5.1) — `lhs` is equal to `rcopy`, unless `rhs` is a non-const xvalue that refers to `lcopy`.

This only holds if all the `doubles` happen to be whole numbers, which is not the case for our example. This invocation would be violating the semantic constraints of the algorithm.

§ 3.2 Return the result of the initial invocation

When we talk about the mathematical definition of fold, that's $f(f(f(f(\text{init}, x_1), x_2), \dots), x_n)$. If we actually evaluate this expression in this context, that's $((1 + 0.25) + 0.75)$ which would be 2.0 .

We cannot in general get this type correctly. A hypothetical f could actually change its type every time which we cannot possibly implement, so we can't exactly mirror the mathematical definition regardless. But let's just put that case aside as being fairly silly.

We could at least address the gotcha from `std::accumulate` by returning the decayed result of invoking the binary operation with T (the initial value) and the reference type of the range. That is, $U = \text{decay_t}<\text{invoke_result_t}<F\&, T, \text{ranges::range_reference_t}<R>>>$. There are two possible approaches to implementing a fold that returns U instead of T :

Two invocation kinds	One invocation kind
<pre>template <range R, movable T, typename F, typename U = /* ... */> auto fold(R&& r, T init, F f) -> U { ranges::iterator_t<R> first = ranges::begin(r); ranges::sentinel_t<R> last = ranges::end(r); if (first == last) { return move(init); } U accum = invoke(f, move(init), *first); for (++first; first != last; ++first) { accum = invoke(f, move(accum), *first); } return accum; }</pre>	<pre>template <range R, movable T, typename F, typename U = /* ... */> auto fold(R&& r, T init, F f) -> U { ranges::iterator_t<R> first = ranges::begin(r); ranges::sentinel_t<R> last = ranges::end(r); U accum = std::move(init); for (; first != last; ++first) { accum = invoke(f, move(accum), *first); } return accum; }</pre>

Either way, our set of requirements is:

- `invocable<F&, T, range_reference_t<R>>` (even though the implementation on the right does not actually invoke the function using these arguments, we still need this to determine the type U)
- `invocable<F&, U, range_reference_t<R>>`
- `convertible_to<T, U>`
- `assignable_from<U&, invoke_result_t<F&, U, range_reference_t<R>>>`

While the left-hand side also needs `convertible_to<invoke_result_t<F&, T, range_reference_t<R>>, U>`.

This is a fairly complicated set of requirements.

But it means that our example, `fold(v, 1, std::plus())` yields the more likely expected result of `2.0`. So this is the version this paper proposes.

§ 4 Other fold algorithms

[P2214R0] proposed a single fold algorithm that takes an initial value and a binary operation and performs a *left* fold over the range. But there are a couple variants that are also quite valuable and that we should adopt as a family.

§ 4.1 fold_first

Sometimes, there is no good choice for the initial value of the fold and you want to use the first element of the range. For instance, if I want to find the smallest string in a range, I can already do that as `ranges::min(r)` but the only way to express this in terms of `fold` is to manually pull out the first element, like so:

```
auto b = ranges::begin(r);
auto e = ranges::end(r);
ranges::fold(ranges::next(b), e, *b, ranges::min);
```

But this is both tedious to write, and subtly wrong for input ranges anyway since if the `next(b)` is evaluated before `*b`, we have a dangling iterator. This comes up enough that this paper proposes a version of `fold` that uses the first element in the range as the initial value (and thus has a precondition that the range is not empty).

This algorithm exists in Rust (under the name `fold_first` as a nightly-only experimental API and `fold1` in the `Itertools` crate) and Haskell (under the name `foldl1`) and Scala and Kotlin (which call it `reduce`, while the version that takes an initial value is called `fold`).

The question is: should we give this algorithm a different name (e.g. `fold_first`) or provide a distinct overload of `fold`? To answer that question, we have to deal with the question of ambiguity. For two arguments, `fold(xs, a)` can only be interpreted as a `fold` with no initial value using `a` as the binary operator. For four arguments, `fold(xs, a, b, c)` can only be interpreted as a `fold` with `a` as the initial value, `b` as the binary operation that is the reduction function, and `c` as a unary projection.

What about `fold(xs, a, b)`? It could be:

1. Using `a` as the initial value and `b` as a binary reduction of the form `(A, X) -> A`.
2. Using `a` as a binary reduction of the form `(X, Y) -> X` and `b` as a unary projection of the form `x -> Y`.

Is it possible for these to collide? It would be an uncommon situation, since `b` would have to be both a unary and a binary function. But it is definitely *possible*:

```
inline constexpr auto first = [](auto x, auto... ){ return x; };
auto r = fold(xs, first, first);
```

This call is ambiguous! This works with either interpretation. It would either just return `first` (the lambda) in the first case or the first element of the range in the second case, which makes it either completely useless or just mostly useless.

There might be a situation that is actually useful in which there is an ambiguity in these cases. But if one arises, it is fairly straightforward to force the correct interpretation by coercing the last argument to be either a binary or unary function:

```
#define FWD(x) static_cast<decltype(x)&&>(x)
#define RETURNS(e) -> decltype((e)) { return e; }
inline constexpr auto as_unary = [](auto f){ return [=](auto&& x) RETURNS(f(FWD(x))); };
inline constexpr auto as_binary = [](auto f){ return [=](auto&& x, auto&& y) RETURNS(f(FWD(x), FWD(y))); };

fold(xs, first, as_binary(first)); // definitely interpretation #1
fold(xs, first, as_unary(first)); // definitely interpretation #2
```

As such, this paper proposes an overload for `fold` that take no initial value (and have a precondition that the range is non-empty) rather than introducing a different name for this case.

§ 4.2 `fold_right`

While ranges::`fold` would be a left-fold, there is also occasionally the need for a *right*-fold. While a `fold_right` is much easier to write in code given `fold` than `fold_first`, since `fold_right(r, init, op)` is `fold(r | views::reverse, init, flip(op))`, it's sufficiently common that it may as well be in the standard library.

As with the previous section, we should also provide overloads of `fold_right` that do not take an initial value.

There are three questions that would need to be asked about `fold_right`.

First, the order of operations of to the function. Given `fold_right([1, 2, 3], z, f)`, is the evaluation `f(1, f(2, f(3, z)))` or is the evaluation `f(f(f(z, 3), 2), 1)`? Note that either way, we're choosing the 3 then 2 then 1, both are right folds. It's a question of if the initial element is the left-hand operand (as it is in the left fold) or the right-hand operand (as it would be if consider the right fold as a flip of the left fold). For instance, Scheme picks the former but Haskell picks the latter.

One advantage of the former is that we can specify `fold_right(r, z, op)` as precisely `fold_left(views::reverse(r), z, op)` and leave it at that. With the latter, we would need need slightly more specification and would want to avoid saying `flip(op)` since directly invoking the operation with the arguments in the correct order is a little better in the case of ranges of prvalues.

This paper picks the latter (that is `fold_right` as the order of arguments flipped from `fold`).

Second, supporting bidirectional ranges is straightforward. Supporting forward ranges involves recursion of the size of the range. Supporting input ranges involves recursion and also copying the whole range first. Are either of these worth supporting? The paper simply supports bidirectional ranges.

Third, the naming question.

§ 4.3 Naming

There are roughly three different choices that we could make here:

1. Provide the algorithms `fold` (a left-fold) and `fold_right`.
2. Provide the algorithms `fold_left` and `fold_right`.
3. Provide the algorithms `fold_left` and `fold_right` and also provide an alias `fold` which is also `fold_left`.

There's language precedents for any of these cases. F# and Kotlin both provide `fold` as a left-fold and suffixed right-fold (`foldBack` in F#, `foldRight` in Kotlin). Elm, Haskell, and OCaml provide symmetrically named algorithms (`foldl/foldr` for the first two and `fold_left/fold_right` for the third). Scala provides a `foldLeft` and `foldRight` while also providing `fold` to also mean `foldLeft`.

In C++, we don't have precedent in the library at this point for providing an alias for an algorithm, although we do have precedent in the library for providing an alias for a range adapter (keys and values for `elements<0>` and `elements<1>`, and [\[P2321R0\]](#) proposes `pairwise` and `pairwise_transform` as aliases for `adjacent<2>` and `adjacent_transform<2>`). We also have precedent in the library for asymmetric names (`sort` vs `stable_sort` vs `partial_sort`), although those algorithms are not as symmetric as `fold_left` and `fold_right`... while we do also have `shift_left` and `shift_right`.

All of which is to say, I don't think there's a clear answer to this question. I would be quite happy with any of the three options. This paper picks (2).

§ 5 Wording

Append to 25.4 [\[algorithm.syn\]](#):

```
#include <initializer_list>

namespace std {
    // ...

    // [alg.fold], folds
    namespace ranges {
        template<class F>
        struct flipped { // exposition only
            F f;

            template<class T, class U>
                requires invocable<F&, U, T>
                constexpr invoke_result_t<F&, U, T> operator()(T&&, U&&);
        };
    };
}
```

```

};

template <class F, class T, class I, class U>
concept indirectly-binary-left-foldable-impl = // exposition only
    movable<T> &&
    movable<U> &&
    convertible_to<T, U> &&
    invocable<F&, U, iter_reference_t<I>>> &&
    assignable_from<U&, invoke_result_t<F&, U, iter_reference_t<I>>>>;

template <class F, class T, class I>
concept indirectly-binary-left-foldable = // exposition only
    copy_constructible<F> &&
    indirectly_readable<I> &&
    invocable<F&, T, iter_reference_t<I>>> &&
    convertible_to<invoke_result_t<F&, T, iter_reference_t<I>>>,
        decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>>> &&
    indirectly-binary-left-foldable-impl<F, T, I,
        decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>>>;

template <class F, class T, class I>
concept indirectly-binary-right-foldable = // exposition only
    indirectly-binary-left-foldable<flipped<F>, T, I>;

template<input_iterator I, sentinel_for<I> S, class T, class Proj =
    identity,
    indirectly-binary-left-foldable<T, projected<I, Proj>>> F>
constexpr auto fold_left(I first, S last, T init, F f, Proj proj = {});

template<input_range R, class T, class Proj = identity,
    indirectly-binary-left-foldable<T, projected<iterator_t<R>, Proj>>> F>
constexpr auto fold_left(R&& r, T init, F f, Proj proj = {});

template <input_iterator I, sentinel_for<I> S, class Proj = identity,
    indirectly-binary-left-foldable<iter_value_t<I>, projected<I, Proj>>>
    F>
    requires constructible_from<iter_value_t<I>, iter_reference_t<I>>>
constexpr auto fold_left(I first, S last, F f, Proj proj = {});

template <input_range R, class Proj = identity,
    indirectly-binary-left-foldable<range_value_t<R>,
    projected<iterator_t<R>, Proj>>> F>
    requires constructible_from<range_value_t<I>, range_reference_t<I>>>
constexpr auto fold_left(R&& r, F f, Proj proj = {});

template<bidirectional_iterator I, sentinel_for<I> S, class T, class
    Proj = identity,
    indirectly-binary-right-foldable<T, projected<I, Proj>>> F>
constexpr auto fold_right(I first, S last, T init, F f, Proj proj = {});

template<bidirectional_range R, class T, class Proj = identity,
    indirectly-binary-right-foldable<T, projected<iterator_t<R>, Proj>>> F>
constexpr auto fold_right(R&& r, T init, F f, Proj proj = {});

template <bidirectional_iterator I, sentinel_for<I> S, class Proj =
    identity,

```

```

    indirectly-binary-right-foldable<iter_value_t<I>, projected<I, Proj>>
        F>
    requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto fold_right(I first, S last, F f, Proj proj = {});

template <bidirectional_range R, class Proj = identity,
    indirectly-binary-right-foldable<range_value_t<R>,
        projected<iterator_t<R>, Proj>> F>
    requires constructible_from<range_value_t<I>, range_reference_t<I>>
constexpr auto fold_right(R&& r, F f, Proj proj = {});
}
}

```

And add a new clause, [alg.fold]:

```

template<input_iterator I, sentinel_for<I> S, class T, class Proj =
    identity,
    indirectly-binary-left-foldable<T, projected<I, Proj>> F>
constexpr auto ranges::fold_left(I first, S last, T init, F f, Proj proj =
    {});

template<input_range R, class T, class Proj = identity,
    indirectly-binary-left-foldable<T, projected<iterator_t<R>, Proj>> F>
constexpr auto ranges::fold_left(R&& r, T init, F f, Proj proj = {});

```

1 *Effects*: Equivalent to:

```

using U = invoke_result_t<F&, T, indirect_result_t<Proj&, I>>;
if (first == last) {
    return U(std::move(init));
}

U accum = invoke(f, std::move(init), invoke(proj, *first));
for (++first; first != last; ++first) {
    accum = invoke(f, std::move(accum), invoke(proj, *first));
}
return accum;

```

```

template <input_iterator I, sentinel_for<I> S, class Proj = identity,
    indirectly-binary-left-foldable<iter_value_t<I>, projected<I, Proj>> F>
    requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto ranges::fold_left(I first, S last, F f, Proj proj = {});

template <input_range R, class Proj = identity,
    indirectly-binary-left-foldable<range_value_t<R>, projected<iterator_t<R>,
        Proj>> F>
    requires constructible_from<range_value_t<I>, range_reference_t<I>>
constexpr auto ranges::fold_left(R&& r, F f, Proj proj = {});

```

2 *Preconditions*: first != last is true.

3 *Effects*: Equivalent to `return ranges::fold_left(ranges::next(std::move(first)), std::move(last), iter_value_t<I>(*first), f, proj)` except ensuring that the initial value is constructed before the iterator is incremented if first is an input iterator.


```
template<bidirectional_iterator I, sentinel_for<I> S, class T, class Proj =
    identity,
    indirectly-binary-right-foldable<T, projected<I, Proj>> F>
constexpr auto ranges::fold_right(I first, S last, T init, F f, Proj proj =
    {});

template<bidirectional_range R, class T, class Proj = identity,
    indirectly-binary-right-foldable<T, projected<iterator_t<R>, Proj>> F>
constexpr auto ranges::fold_right(R&& r, T init, F f, Proj proj = {});
```

- 4 *Effects*: Equivalent to:

```
using U = invoke_result_t<F&, indirect_result_t<Proj&, I>, T>;

if (first == last) {
    return U(std::move(init));
}

I tail = ranges::next(first, last);
U accum = invoke(f, invoke(proj, *--tail), std::move(init));
while (first != tail) {
    accum = invoke(f, invoke(proj, *--tail), std::move(accum));
}
return accum;
```

```
template <bidirectional_iterator I, sentinel_for<I> S, class Proj =
    identity,
    indirectly-binary-right-foldable<iter_value_t<I>, projected<I, Proj>> F>
requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto ranges::fold_right(I first, S last, F f, Proj proj = {});

template <bidirectional_range R, class Proj = identity,
    indirectly-binary-right-foldable<range_value_t<R>,
    projected<iterator_t<R>, Proj>> F>
requires constructible_from<range_value_t<I>, range_reference_t<I>>
constexpr auto ranges::fold_right(R&& r, F f, Proj proj = {});
```

- 5 *Preconditions*: `first != last` is `true`.

- 6 *Effects*: Equivalent to:

```
I tail = ranges::prev(ranges::next(first, std::move(last)));
return ranges::fold_right(std::move(first), tail, iter_value_t<I>
    (*tail), f, proj);
```

§ 6 References

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.

<https://wg21.link/p2214r0>

[P2321R0] Tim Song. 2021-02-21. zip.

<https://wg21.link/p2321r0>

[P2322R0] Barry Revzin. 2021-02-18. ranges::fold.
<https://wg21.link/p2322r0>

[P2322R1] Barry Revzin. 2021-03-17. ranges::fold.
<https://wg21.link/p2322r1>